

情境检索与冲突消解 Situation Retrieval & Conflict Resolution

原文：Dominik Heckmann 《Ubiquitous User Modeling》第 6 章（书页 119-140，共 22 页），萨尔兰大学博士论文，2006 年出版。这一章是「同一个维度上多条矛盾陈述谁赢」这个问题的完整答案。

说明：原文含 18 张图（流程图、算法伪代码、架构图）。本文档翻译正文，图的内容用文字概述并标明是原文哪一张图。

核心内容

Q：这一章解决什么问题？ A：任何用户和任何系统都可以往库里写陈述，所以库里必然有互相矛盾的内容。这一章回答两件事：怎么把矛盾的陈述找出来（冲突检测），以及找出来之后返回哪一条（冲突消解）。

Q：最根本的那个架构取向是什么？ A：不消解冲突，把矛盾留在库里，由每个消费方各自选策略。原文说得很直白：情境模型可能充满不一致，而这些不一致一般不需要被解决，因为我们不打算把陈述库变成一个前后一致的世界模型。已经收集和传播出去的陈述不会被修改或删除。理由是在高度分布的架构里，信息被层层传递，多数情况下我们没有编辑那些副本的权限。

允许做的只有三件事：把一条陈述具体化成一条新陈述来对它评论；把一条陈述标记为「已被替换」；用隐私机制把它隐藏起来。

这样做的第一个好处是：**每个用户、每个请求方能各自选择偏好的消解策略，从而得到各自的模型。**

Q：查询分成哪几步？ A：四个宏步骤。选择（select）决定查哪个库；匹配（match）返回所有符合匹配属性的陈述；过滤（filter）滤掉更多不想要的；控制（control）执行冲突消解并把最终陈述变成返回结果。查询的属性也按这四步分成三组：匹配属性、过滤属性、控制属性。

Q：匹配属性有什么特别的？ A：两点。第一，陈述模型里的任何一个属性都能当匹配属性用，所以匹配属性的名字和陈述属性的名字完全一样，靠点号区分（`statement.subject` 与 `query.subject`）。第二，匹配不只是相等比较，可以用语义函数——比如匹配条件写 `query.subject = extension(person)`，那么所有主语是 `person` 这个类（及其子类）的实例的陈述都会被匹配上。

Q：过滤器有哪几个？ A：三个。隐私过滤器不需要激活，它永远开着；如果查询里没写请求方和用途，默认取「请求方=匿名、用途=商业」，这个组合导致的是最小访问权限。置信度过滤器由 `minConfidence` 和 `maxConfidence` 激活，用来把置信度限制在一个区间。时间过滤器用 `fromTime` 和 `untilTime`，之所以需要它，是因为这套方法里任何陈述都不删除，只标记为已替换或已过期，所以要靠时间过滤器忽略旧陈述，或者反过来做历史视图。

Q：消解器有哪些，怎么组合？ A：十二个消解器，分三类：`most(n)` 类用元数据做决定；`add` 类把已过期或已被替换的陈述加回冲突集；`return` 类不用任何数据做选择。一个有序的消解器列表就是一个策略，写在 `query.strategy` 属性里，用竖线连接。

Q：消解器带来的不确定性怎么处理？ A：这是这一章最该抄的一条机制。原文说：消解器和策略隐含着不确定性，为此要相应地修改结果陈述的置信度值，并且把这次冲突的情形写进 `evidence` 属性。也就是说，消解不是悄悄挑一条返回，消解这个动作本身要留痕。

Q：冲突怎么检测出来？ A：分成六级，按计算负载递增。第 0 级只看主语、辅助词、谓词、取值域四项是否相同，纯语法比较。第 1 级在第 0 级基础上去掉已过期和已被替换的。第 2 级允许取值域不同，靠刻度映射对齐。第 3 级允许谓词不同，靠同义词、上位词、整体词、反义词把相关谓词归为一类。第 4 级允许辅助词也不同。第 5 级允许主语不同，处理「组与成员」以及「同一个人在不同系统里有不同账号」。

Q：这套东西的诚实局限是什么？ A：原文自己列了两条。第一，不同的策略会给出完全不同的答案，而「哪个策略才是对的」并不显然。第二，冲突消解只能和可用的陈述集一样好——如果冲突集里有几条陈述根本没带置信度值，置信度过滤器就无从应用；时间约束没给，时效过滤器同样无从应用。

背景补充 / 点评

先交代两个术语的来历。

产生式系统（production system）里的「冲突集」（conflict set）是这一章方法的直接来源，原文明说借用了这个技术，并引用了 OPS5 这个 1980 年代的产生式系统。产生式系统的工作方式是：一堆「如果…那么…」规则，每一轮找出所有条件成立的规则，这些规则构成冲突集，然后按一套元规则从冲突集里挑一条执行。这一章把这个机制搬到了陈述上：找出所有互相矛盾的陈述构成冲突集，再按策略挑。

怀疑式继承（sceptical inheritance）是非单调推理里的一个立场。当一个对象从多条继承路径得到互相矛盾的属性时，怀疑式的做法是**什么都不结论**，与之相对的轻信式（credulous）做法是随便选一条。原文用这个概念来解释 `returnNone` 这个消解器的性质。

我的判断：这一章是六篇文献里对我们最直接有用的一篇，因为它回答的正是会上没有结论的那批问题。而它的价值分三层，一层比一层重要。

最表层的价值是那十二个消解器，可以当清单直接用。中间层的价值是那六级冲突检测——它把「什么叫两条陈述在说同一件事」这个模糊问题，变成了一个有六个明确档位的判断，每一档的计算代价和覆盖能力都写清楚了。

最深的价值是那个架构取向：**不追求库内一致，把选择权交给消费方**。这一条反直觉，但理由很硬。传统做法是入库时就消解，库里永远只有一份「正确答案」。这么做的隐含假设是「存在一个对所有消费方都正确的答案」。而现实是不同消费方要的不一样——做教学适配时你要最可信的那条，做学员自述展示时你要学员自己说的那条，做变化轨迹分析时你要全部。入库消解会把后两种需求直接杀掉，而且杀得没有痕迹。

那个下棋的例子把这一点演示得很干净：三条关于同一个人棋力的矛盾陈述，五种策略跑出来，第一种返回全部三条，第二和第三种返回「高」，第四种返回「中」，第五种返回「低」。**同一批数据，四个不同答案，而且每一个都有道理。**如果入库时就消解，我们只会看到其中一个，并且以为那就是事实。

留白和该打折的地方也说清楚。第 4 级冲突检测（辅助词语义相关）原文明说「尚未用于具体的冲突消解」，也就是没实现。第 5 级依赖组成员映射和跨系统身份对齐，这两件事本身都是难题。语义冲突检测那部分依赖同义词上位词这类词汇关系，原文靠的是 WordNet，我们中文场景没有等价资源，这一块要重新想办法。另外原文留了一个开放问题：最佳策略与维度的等价类之间是否存在相关性？如果存在，这个信息可以写进本体，用一层元规则来解决「该选哪个策略」。**这个开放问题二十年了，据我所知仍然开放。**

对我自己的启示，有六条。

第一条，**「不消解、留给消费方选」这个取向要在设计第一天就定下来**，因为它决定了存储结构。会上已经定了「保留全部版本、软删除」，方向是对的，但会上给的理由是「变化轨迹有业务价值」。这一章给的理由更根本：不存在一个对所有消费方都正确的答案。这两个理由推出的设计不完全一样——按前者，只要留着历史就行；按后者，**消解策略必须是查询的一个参数**，不能写死在服务里。

第二条，**十二个消解器里有五个我们立刻就要**：`mostRecent`（最近的）、`mostConfident`（最可信的）、`mostPersonal`（学员自己说的优先）、`returnAll`（全给，用于变化轨迹）、`returnNone`（拿不准就什么都不返回）。其中 `mostPersonal` 那条的理由值得注意：原文承认「用户可能不是自己最好的判断者」，但出于隐私考虑，用户主动给出的陈述应当被优先。这是一个把隐私放在准确性前面的取舍，我们要不要跟，得单独决定。

第三条，**`returnNone` 就是「不静默降级」的学术版**，而且原文给了它一个更好的名字：这是一种「不说错误的非常安全的方式」，等于说「我不知道」。会上讨论谄媚问题时，缺的正是这个——一个明确的「我判断不出来，所以不说」的返回值。

第四条，**消解要改置信度、要写 evidence，这条机制必须抄**。会上讨论 trace 时说的是「每轮用了哪种提取方式、增了什么减了什么」，那是抽取侧的留痕。这一条补的是消费侧的留痕：一条画像被消费时，如果它是从三条矛盾陈述里挑出来的，那么这次挑选的过程要写进结果。不写的话，下游看到的是一条干干净净的陈述，完全不知道它的兄弟被丢掉了。

第五条，**六级冲突检测可以直接当我们的实现路线图**。第 1 级（同主语同辅助词同谓词同取值域，去掉过期与被替换）是必须做的，纯字符串比较，成本极低。第 2 级（刻度映射）是第二步，前提是刻度已经独立登记（对应 03 文档借鉴点 8）。第 3 级往上先不做——中文缺 WordNet 这种资源，而且原文自己也没跑通第 4 级。明确写下「我们只做到第 2 级」比含糊地说「支持语义冲突检测」诚实得多。

第六条要警惕的：**`strategy` 和 `ranking` 用同一批词但作用不同，这是个容易混的设计**。原文明确指出这一点——`strategy=mostRecent` 是「只留最近的」（筛掉别的），`ranking=mostRecent` 是「按时间排序」（一条不筛）。我们如果照抄这套命名，要在文档里把这个区别写死，否则用的人一定会搞错。

"We do not intend to change the statement repositories into one consistent world model."
我们并不打算把陈述库变成一个前后一致的世界模型。

"These completely different results show the power of the conflict resolution strategies, however they also indicate that not all conflicting statements can be satisfactorily solved by resolution strategies." 这些完全不同的结果既显示了冲突消解策略的威力，也表明并非所有互相矛盾的陈述都能靠消解策略得到令人满意的解决。

"An important issue to keep in mind is the problem that resolvers and strategies imply uncertainty. To contribute to this, the confidence value of the resulting statement is appropriately changed, furthermore the conflict situation is added to the evidence attribute." 有一个重要问题要牢记：消解器与策略隐含着不确定性。为此，结果陈述的置信度值会被相应地修改，此外这次冲突的情形会被写进 evidence 属性。

"This is a very safe way not to say something wrong. This rule could be compared with sceptical inheritance in non-monotonic reasoning: I don't know!" 这是一种不说错话的、非常安全的方式。这条规则可以类比非单调推理里的怀疑式继承：我不知道！

"The conflict resolution can only be as good as the available set of statements." 冲突消解的好坏，只能与可用的那批陈述一样好。

"The privacy filter needs no activation since it is always activated." 隐私过滤器不需要激活，因为它一直是激活状态。

全文翻译

开篇

The newly defined term of situation retrieval describes specialized information retrieval within repositories and reports of SITUATIONAL STATEMENTS. In chapter 4 we have demonstrated that the model of the statements is independent from the actual used syntactical realization and in chapter 5 we have demonstrated that the model is even decoupled from the semantics which is moved to the ontology. For the information retrieval part this insight leads to the threefold relationship: one with the model, one with the syntax and one with the semantics.

我们新定义的「情境检索」这个术语，指的是在「情境陈述」的库与报告之内所做的专门化信息检索。第 4 章我们已经论证过，陈述的模型与实际采用的语法实现无关；第 5 章我们又论证过，这个模型甚至与语义也是解耦的——语义被移进了本体。对信息检索这部分而言，这个洞察引出三重关系：一重与模型有关，一重与语法有关，一重与语义有关。

Since context logs, user journals and especially user models are also represented in this statement framework, situation retrieval directly covers the task of querying these logs, journals and models.

由于上下文日志、用户日志、尤其是用户模型也都用这套陈述框架来表示，情境检索直接覆盖了「查询这些日志、日志册与模型」这项任务。

6.1 情境查询的模型

SITUATIONAL QUERIES form the counterpart to SITUATIONAL STATEMENTS, since each situation attribute finds a corresponding attribute in the model of SITUATIONAL QUERIES. SITUATIONAL QUERIES consist of three identified groups of attributes: the match-attributes, the filter-attributes and the control-attributes. They are named after three macro-steps in the query evaluation process.

「情境查询」是「情境陈述」的对应物，因为每一个情境属性在情境查询的模型里都能找到一个对应的属性。情境查询由三组已识别的属性构成：匹配属性、过滤属性、控制属性。它们的命名来自查询求值过程中的三个宏步骤。

6.1.1 匹配盒

The match-attributes are named subject, auxiliary, predicate, range, object and so on, exactly after their corresponding attributes within the SITUATIONAL STATEMENTS. Actually any attribute from the model of statements can be used as match attribute in the model of queries. To differentiate between the two sets, the dot-notation is introduced as in `statement.subject` versus `query.subject`. When the matching algorithm is applied, only statements that match all given match-attributes are handed over to the filtering algorithm.

匹配属性的名字是主语、辅助词、谓词、取值域、宾语等等，与「情境陈述」里对应的属性完全同名。实际上陈述模型里的任何一个属性，都能在查询模型里当作匹配属性使用。为了区分这两套同名的东西，引入点号记法，例如 `statement.subject` 与 `query.subject`。匹配算法执行时，只有那些**满足全部给定匹配属性**的陈述，才会被交给过滤算法。

Furthermore the matching algorithm can be more complex than just the comparison for equality: semantic functionality like the extension relation can be applied. This relation maps to each class its subsumed instances. For example if the match attribute `query.subject = extension(person)` is given, all statements with `statement.subject` in `{Boris, Jörg, Margeritta, ...}` are matched, namely instances that belong to the class (or the subclasses) of person. This

elegant mechanism to integrate semantic inference into the syntax of the query language has already been prepared by the extended RDF resource concept.

此外，匹配算法可以比单纯的相等比较更复杂：可以运用「外延关系」这类语义功能。这个关系把每一个类映射到它所涵盖的实例。举例来说，如果匹配属性写成 `query.subject = extension(person)`，那么所有主语属于 {Boris、Jörg、Margeritta……} 这些实例的陈述都会被匹配上，也就是那些属于 person 这个类（或其子类）的实例。这个把语义推断优雅地整合进查询语言语法的机制，已经由扩展后的 RDF 资源概念做好了准备。

6.1.2 过滤盒

Filter attributes set further restrictions on the returned statements. There are currently three filters implemented: a privacy filter, a confidence filter and a temporal filter. The attributes `query.requestor` and `query.intention` decide according to the attributes `statement.access`, `statement.purpose` and `statement.owner` if the privacy filter can be passed. If no permission is granted, the statement will be filtered out, even though it might contain the answer to the query.

过滤属性对返回的陈述施加进一步的限制。目前实现了三个过滤器：隐私过滤器、置信度过滤器、时间过滤器。`query.requestor`（请求方）和 `query.intention`（意图）这两个属性，会依据陈述上的 `access`（访问权）、`purpose`（用途）、`owner`（所有者）三个属性，来决定隐私过滤器能否通过。如果没有获得许可，**这条陈述就会被滤掉——即使它可能正好包含这个查询的答案。**

The privacy filter needs no activation since it is always activated. If no requestor and purpose attributes are stated in the query, the values `requestor=anonymous` and `purpose=commercial` are set as defaults, which results in a minimal access in privacy terms.

隐私过滤器不需要激活，因为它一直是激活状态。如果查询里没有写请求方和用途这两个属性，就取默认值「请求方=匿名」和「用途=商业」，这个组合在隐私意义上导致的是最小的访问权限。

The confidence attributes `minConfidence` and `maxConfidence` allow us to restrict the confidence value to a certain interval. For example it might be interesting to analyze only statements that carry a higher confidence value than 0.75 within the range [0, 1]. The confidence filter will be used automatically, if at least one of the attribute-value pairs is used.

置信度属性 `minConfidence` 与 `maxConfidence` 让我们能把置信度值限制在某个区间内。举例来说，也许我们只想分析那些在 0 到 1 的取值范围里、置信度高于 0.75 的陈述。只要用到了这两个属性中的至少一个，置信度过滤器就自动生效。

The temporal filter allows to put constraints on the statements' temporal aspects, since in our approach no statement is deleted, but either marked as replaced or marked as expired if it should not be used any more. This especially allows to neglect older statements with the attribute `fromTime`, or the other way round, to allow for historic views by filtering out all statements after a certain point of time with the attribute `untilTime`.

时间过滤器允许对陈述的时间性质施加约束。之所以需要它，是因为在我们这套方法里**没有任何陈述会被删除**：一条陈述如果不该再被使用，就被标记为「已被替换」或「已过期」。这尤其使得我们能用 `fromTime` 属性忽略较旧的陈述；或者反过来，用 `untilTime` 属性滤掉某个时间点之后的全部陈述，从而得到历史视图。

6.1.3 控制盒

The attribute repository controls the select-step by defining the repository (or the set of repositories) to which the query is being applied to. During the control-step, possible conflicts are resolved and the appearance of the returned UserML document is defined. The conflict resolution can be guided by the attributes strategy and ranking and the appearance of the result is influenced by the two attributes format and naming. The attribute function allows to post-process the so far calculated set of statements into any kind of result like an average number for example.

`repository`（库）这个属性通过指定「这个查询要作用在哪个库、或哪几个库上」，来控制选择这一步。在控制这一步里，可能存在的冲突被消解，并且返回文档的呈现方式被确定。冲突消解由 `strategy`（策略）和 `ranking`（排序）两个属性引导，结果的呈现方式受 `format`（格式）和 `naming`（命名）两个属性影响。`function`（函数）这个属性允许把到此为止算出的那批陈述做后处理，变成任何形式的结果，比如一个平均数。

6.1.4 情境请求的模型

A list of SITUATIONAL QUERIES is called a SITUATION REQUEST. A SITUATION REQUEST is sent to the user model and context service that resolves each query after the other in a row (or if possible in parallel) and returns the resulting SITUATION REPORT.

一组情境查询的列表叫做一个「情境请求」。情境请求被发给用户模型与上下文服务，该服务依次（或者在可能时并行地）处理其中每一个查询，并返回结果，即一份「情境报告」。

（原文表 6.1 给出情境查询的全部属性及其默认值。匹配组：主语、辅助词、谓词、取值域、宾语、编号、分组、位置等，**它们的默认值全都是「任意」，唯有分组的默认值是「用户模型」**；说明里写着「为情境陈述引入的所有属性都可以用作匹配属性」。过滤组：请求方默认匿名；意图默认商业；最小置信度默认 0；最大置信度默认 1；起始时间默认「无论何时」；结束时间默认「现在」。控制组：库默认由系统选择；**策略默认 latestOnly（只要最新的）；排序默认 newestFirst（最新在前）**；命名默认长名；格式默认 UserML；函数默认无。）

6.1.5 查询应答算法

The select-step chooses the report or repository to which the query is applied. The match-step returns all statements that match the corresponding query attributes. The filter-step filters out

further unwanted statements, while the control-step performs conflict-resolution and transforms the final statements into the returned result.

选择这一步挑出这个查询要作用的报告或库。匹配这一步返回所有符合相应查询属性的陈述。过滤这一步滤掉更多不需要的陈述，而控制这一步执行冲突消解，并把最终的陈述变成返回的结果。

（原文图 6.3 给出应答算法的伪代码，是四步的线性串联：先由用户模型与上下文服务执行选择过程，然后应答过程把剩下三个宏步骤线性化，每一步都返回一个更小的陈述集合。）

6.3 冲突消解

In widely spread distributed user modeling and context awareness one can not expect that all systems use the same standards for representing user models and context information. However, as a common basis and to narrow down the research area, we assume that all systems that are involved in the information exchange are able to use the framework of SITUATIONAL STATEMENTS.

在广泛分布的用户建模与上下文感知中，不能指望所有系统都用同一套标准来表示用户模型和上下文信息。不过，作为共同基础、并且为了收窄研究范围，我们假定所有参与信息交换的系统都能使用「情境陈述」这套框架。

6.3.1 情境冲突的分类

As every user and every system is allowed to enter statements into repositories, some of this information might be contradictory. Conflicts among SITUATIONAL STATEMENTS like for example a contradiction caused by different opinions of different creators or changed values over time are categorized in the following listing.

由于每个用户和每个系统都被允许往库里写入陈述，这些信息中有一些可能互相矛盾。情境陈述之间的冲突——比如由不同创建者的不同看法造成的矛盾，或者取值随时间改变造成的矛盾——按下面的清单分类。

1. ON THE REPRESENTATIONAL LEVEL: each system can choose between a variety of possible representations to express the same information which leads to the so-called variation mapping.

一、**表示层面**：每个系统都可以在多种可能的表示方式中挑一种来表达同一份信息，这导致了所谓的「变体映射」问题。

2. ON THE SYNTACTICAL LEVEL: statements can for instance differ in the use of the statement attributes like subject, predicate, object, range, start etc., clear modeling guidelines are necessary.

二、**语法层面**：陈述在使用主语、谓词、宾语、取值域、开始时间等属性上可能各不相同，因此**清晰的建模指南是必需的**。

3. ON THE SEMANTICAL LEVEL: the systems are not forced to use the same vocabulary, that is to say the same ontology, to represent the meaning of the concepts, which leads to the user model integration problem number one: ontology merging and semantic web integration.

三、**语义层面**：系统并不被强制使用同一套词汇表（也就是同一个本体）来表示概念的含义，这引出了用户模型集成的头号问题：本体合并与语义网集成。

4. ON THE OBSERVATION AND INFERENCE LEVEL: several sensors can see same things differently and claim to be right, measurement errors can occur, systems may have preferred information sources.

四、**观测与推断层面**：多个传感器可能对同一件事看法不同，而且各自都声称自己是对的；测量误差会发生；不同系统可能有各自偏好的信息来源。

5. ON THE TEMPORAL AND SPATIAL LEVEL: information can be out of date or out of spatial range, a degree of expiry can hold, thus reasoning on temporal and spatial meta data becomes necessary.

五、**时间与空间层面**：信息可能已经过时，或者超出了空间适用范围，可能存在某种程度的过期，因此对时间和空间元数据做推理成为必要。

6. ON THE PRIVACY AND TRUST LEVEL: information can be hidden, incomplete, secret or falsified on purpose, a system of trustworthiness could be applied.

六、**隐私与信任层面**：信息可能被隐藏、不完整、保密，或者被故意伪造，可以引入一套可信度机制。

All these aspects are content- and domain independent and can be relevant for user related as well as context related information. In relation to the situation retrieval task, the two concepts of precision (How many retrieved statements are really relevant?) and recall (Is all relevant information retrieved?) are also taken into consideration. Conflict resolution forms the first part of the control macro-step.

以上这些方面都与内容和领域无关，对与用户相关的信息和与上下文相关的信息同样适用。就情境检索这项任务而言，我们还要把精确率（检索出来的陈述里有多少是真正相关的）和召回率（相关信息是否都被检索出来了）这两个概念考虑在内。冲突消解构成控制这个宏步骤的第一部分。

6.3.2 开篇的两个矛盾陈述例子

A) Imagine you should play chess against user A and you have the following rule of thumb: if the opponent is a good player, you start with a defensive opening; but if the opponent is a weak player, you choose an offensive opening. You have heard the three following, conflicting statements about user A's ability to play chess: System C claims that A is a good player, system B claims that A is a medium good player while user A claims that she/he himself is a bad player.

例 A：设想你要和用户 A 下棋，而你有这样一条经验法则：如果对手是好棋手，你就用防守型开局；如果对手是弱棋手，你就选进攻型开局。关于用户 A 的棋力，你听到了三条互相矛盾的陈述：系统 C 声称 A 是好棋手，系统 B 声称 A 是中等棋手，而用户 A 自己声称他棋很差。

（原文图 6.14 把这三条陈述画成三个矩形，并标出元数据的两个维度：时间轴显示开始时间、结束时间、持续性与过期值，由此可见**系统 B 那条是最旧的，系统 C 那条是最新的**；矩形的高度对应置信度值，**系统 B 那条置信度最高，用户自己那条置信度最低**。）

So, how should we decide? How do we resolve such conflicts? Naive approaches could ask: should we return the latest entry? Should we return a random element? Should we return the one with the highest confidence value or should we prefer the user's entry? Conflicting statements turned out to be a complex and general problem but with no general solution that is valid for all situations.

那么我们该怎么决定？怎么消解这类冲突？幼稚的做法会问：要不要返回最新的那条？要不要随机返回一条？要不要返回置信度最高的那条，还是应该优先用户自己填的那条？事实证明，矛盾的陈述是一个复杂而普遍的问题，**而且没有一个对所有情形都成立的通用解法**。

B) Let us assume that a resource-adaptive mobile device uses the following energy saving and contrast optimizing adaptation rule if its device's battery is low: if the surrounding brightness is low then set the display brightness to its minimum, if the surrounding brightness is medium or high then set the display brightness to medium. Since the mobile device has no light sensor by itself, it needs to retrieve and infer information about the surrounding brightness from the user model and context service.

例 B：假设一台资源自适应的移动设备在电量低时采用如下的省电兼对比度优化规则：如果周围亮度低，就把屏幕亮度设到最小；如果周围亮度中等或高，就把屏幕亮度设到中等。由于这台设备自己没有光线传感器，它需要从用户模型与上下文服务那里检索并推断周围亮度的信息。

The current position of the mobile device should be Room124 and the light situation there is given by the semantically conflicting statements: 1) Room124 hasProperty brightness = low. 2) Saarbrücken hasProperty weather = sunny with no clouds. 3) Light124.1 hasProperty switched = on.

设备当前位置是 124 房间，那里的光照情况由以下几条语义上互相矛盾的陈述给出：一、124 房间的亮度是「低」。二、萨尔布吕肯的天气是「晴朗无云」。三、124 房间的 1 号灯的开关状态是「开」。

The most direct statement 1) about the brightness in Room124 claims that it is dark. However, statement 2) claims that the sun is shining. Since the device is inside a building which could hinder the sun to lighten the surroundings the situation is not clear. Furthermore statement 3) claims that the lights in this room are switched on. This conflict is difficult to detect since ontological reasoning, spatial reasoning and qualitative reasoning might be necessary to handle this problem satisfactorily.

关于 124 房间亮度最直接的那条陈述（第一条）说这里是暗的。然而第二条说太阳正在照。由于设备在建筑内部，而建筑可能挡住阳光照亮周围，所以情况并不清楚。此外第三条说这个房间的灯是开着的。**这个冲突很难被检测出来**，因为要令人满意地处理这个问题，可能需要本体推理、空间推理和定性推理。

6.3.3 冲突消解器与消解策略

Conflict Resolvers are a special kind of filter that control the conflict resolution process. An ordered list of these resolvers defines the conflict resolution strategy. They are modeled in the `query.strategy` attribute. These resolvers are needed if the match process and filter process leave several conflicting statements as possible answers. Three kinds of conflict resolvers can be identified: the `most(n)`-resolvers that use meta data for their decision, the `add`-resolvers that add expired or replaced statements to the conflict sets, and the `return`-resolvers that don't use any data for their selection.

冲突消解器是一类特殊的过滤器，用来控制冲突消解过程。**这些消解器的一个有序列表就定义了一个冲突消解策略**，它们写在 `query.strategy` 属性里。当匹配过程和过滤过程之后仍然剩下好几条互相矛盾的陈述作为候选答案时，就需要这些消解器。可以识别出三类消解器：用元数据做决定的 `most(n)` 类；把已过期或已被替换的陈述加回冲突集的 `add` 类；以及不使用任何数据来做选择的 `return` 类。

mostRecent(n): Especially where sensors send new statements on a frequent basis, values tend to change quicker than they expire. This leads to conflicting non-expired statements. The `mostRecent(n)` resolver returns the `n` newest non-expired statements.

最近 n 条：尤其在传感器频繁发送新陈述的场合，取值变化的速度往往快于它们过期的速度，这就导致了一批都没过期却互相矛盾的陈述。这个消解器返回最新的 `n` 条未过期陈述。

mostNamed(n): If there are many statements that claim A and only a few claim B or something else, than `n` of the "most named" statements are returned. Of course it is not certain that the majority necessarily tells the truth but it could be a reasonable rule of thumb for some cases.

说得最多的 n 条：如果有很多条陈述主张 A，而只有少数几条主张 B 或别的，那么就返回「被说得最多」的那 `n` 条。当然，多数并不必然说的就是真话，但在某些情形下这可以是一条合理的经验法则。

mostConfident(n): If the confidence values of several conflicting statements can be compared with each other, it seems to be an obvious decision to return the `n` statements with the highest confidence value.

最可信的 n 条：如果几条互相矛盾的陈述的置信度值彼此可比，那么返回置信度最高的那 `n` 条看起来是个显而易见的决定。

mostSpecific(n): If the range or the object of a statement is more specific than in others, the `n` "most specific" statements are returned. For example if `auxiliary=hasKnowledge`, `predicate=chess` and `first range=yesNo` while the second `range=Novice-Occasional-Professional`

Expert-Grandmaster, the statement with the second range contains a more specific information. Another specificity range ordering is for example: yesNo < lowMediumHigh < 0%-100%.

最具体的 n 条: 如果某条陈述的取值域或宾语比别的更具体, 就返回「最具体」的那 n 条。举例来说, 辅助词是「有知识」、谓词是「国际象棋」, 第一条的取值域是「是/否」, 第二条的取值域是「新手-偶尔玩-职业-专家-大师」, 那么第二条含有更具体的信息。另一个特异性排序的例子是:「是/否」不如「低中高」,「低中高」不如「0% 到 100%」。

mostPersonal(n): If the creator of the statement is the same as the statement's subject (a self-reflecting statement), this statement is preferred. Furthermore, if an is-friend-of relation is defined, statements by friends could be preferred to statements by others. However, this resolver bears the problem, that users might not be their best judge. However due to privacy arguments, the user's own statements that are given (on purpose) should be preferred.

最贴本人的 n 条: 如果一条陈述的创建者与这条陈述的主语是同一个 (即自述型陈述), 这条陈述就被优先。此外, 如果定义了「是某人的朋友」这种关系, 朋友给出的陈述可以比其他人给出的更被优先。不过这个消解器带着一个问题: **用户可能并不是自己最好的判断者。但出于隐私方面的理由, 用户 (有意) 主动给出的自己的陈述应当被优先。**

addExpired: Per default the already expired statements are filtered out. However, if one wants to take them into consideration, the addExpired-resolver adds these statements to the conflict sets.

加回已过期的: 默认情况下已经过期的陈述会被滤掉。但如果想把它们考虑进来, 这个消解器就把这些陈述加回冲突集。

addReplaced: Statements that are marked with the replaced-flag by other statements are also per default filtered out. The addReplaced-resolver brings these statements back into the process.

加回被替换的: 被其他陈述标记了「已被替换」标志的陈述, 默认也会被滤掉、不参与情境检索过程。这个消解器把这些陈述带回流程里。

addPrivate: Statements that do not pass the privacy settings are always filtered out. However, for development, testing and administrative reasons experimental private statements may also be recognized with the addPrivate-resolver.

加回私密的: 通不过隐私设置的陈述总是会被滤掉。不过出于开发、测试和管理方面的原因, 实验性的私密陈述也可以用这个消解器纳入考虑。

returnAll: If the remaining conflict set should not be resolved any further by the integrated mechanism, the resolver returnAll returns all remaining statements that can then be resolved by an external conflict resolution method, resolved by introspection or left unresolved since our approach also allows conflicting extensions in parallel.

全部返回：如果剩下的冲突集不应再由内建机制做进一步消解，这个消解器就返回全部剩余陈述，它们随后可以由外部的冲突消解方法处理、由内省处理，或者干脆不处理——因为我们这套方法也允许矛盾的扩展并行存在。

returnNone: If there still occurs a conflict that could not be resolved until the returnNone resolver is applied, no statement is returned. This is a very safe way not to say something wrong. This rule could be compared with sceptical inheritance in non-monotonic reasoning: I don't know!

什么都不返回：如果直到这个消解器被应用时冲突仍然没有被消解，那就不返回任何陈述。这是一种不说错话的、非常安全的方式。这条规则可以类比非单调推理里的怀疑式继承：我不知道！

returnRandom(n): if after applying several filters still no unique value is found but a unique answer is expected, a random pick will be offered by this resolver. This credulous behavior is selected by the requestor and therefore acceptable.

随机返回 n 条：如果应用了若干过滤器之后仍然找不到唯一的取值，而调用方又期待一个唯一答案，这个消解器就随机挑一条给出。这种轻信式的行为是由请求方自己选的，因此可以接受。

returnDialog: if no unique value is found, an alternative conflict resolution strategy could be clarification by dialog. In some cases an appropriate human-computer dialog will be initiated in this case.

转对话澄清：如果找不到唯一取值，另一种可选的冲突消解策略是通过对话来澄清。在某些情况下，此时会发起一段合适的人机对话。

These conflict resolver rules are based on common sense heuristics. An important issue to keep in mind is the problem that resolvers and strategies imply uncertainty. To contribute to this, the confidence value of the resulting statement is appropriately changed, furthermore the conflict situation is added to the evidence attribute. Further ideas like calculating the average value or the maximum value of the statement's object can be covered by the function attribute.

这些冲突消解器规则基于常识性的启发式。有一个重要问题要牢记：**消解器与策略隐含着不确定性。为此，结果陈述的置信度值会被相应地修改，此外这次冲突的情形会被写进 `evidence` 属性。** 另一些想法，比如计算陈述宾语的平均值或最大值，可以由 `function` 属性来覆盖。

（原文图 6.16 给出五个策略示例：一、全部返回。二、只要最近 1 条。三、最近 4 条里挑最可信的 3 条、再从中随机返回 2 条。四、说得最多的 5 条里挑最可信的 3 条、再挑最具体的 1 条。五、最贴本人的 3 条里挑最可信的 2 条，然后用平均函数算出这两条宾语的平均值。）

The first conflict resolution strategy is the empty one, which means that no conflict resolution will be applied. Note, if the query.strategy attribute is left empty, a default strategy will be applied but not necessarily the returnAll one. The second strategy is adequate for statements

that are frequently renewed by only one sensor. The fifth strategy could be interesting for user model dimensions that do not change over time like personality traits.

第一个策略是空策略，意思是不做任何冲突消解。**注意：如果 `query.strategy` 属性留空，会应用一个默认策略，而那个默认策略不一定就是「全部返回」。**第二个策略适合那种只由一个传感器频繁刷新的陈述。第五个策略对那些不随时间改变的维度（比如人格特质）可能有意思。

If we revisit the prefacing example and apply the strategies, the second and the third strategy would return high, the fourth strategy would return medium while the fifth strategy would return low. However, the first strategy returns all three values but is no help in this situation. These completely different results show the power of the conflict resolution strategies, however they also indicate that not all conflicting statements can be satisfactorily solved by resolution strategies. As different strategies lead to different statements and resulting values, the choice of the "right" conflict resolution strategy isn't that obvious.

如果我们回到开篇那个下棋的例子，把这些策略套上去：第二和第三个策略会返回「高」，第四个策略会返回「中」，而第五个策略会返回「低」。第一个策略把三个值全都返回，但在这个情形下帮不上忙。**这些完全不同的结果既显示了冲突消解策略的威力，也表明并非所有互相矛盾的陈述都能靠消解策略得到令人满意的解决。由于不同策略导致不同的陈述和不同的结果值，究竟哪个才是「对」的冲突消解策略，并不那么显而易见。**

Different classes of problems seem to need different conflict resolution strategies. The open question is if there is any correlation of the best strategy in certain circumstances. We expect that already the user model dimension equivalence classes correlate with the best strategies. If so, this information could be added to the general user model ontology GUMO and the problem could be solved with the introduction of a new level of meta rules.

不同类别的问题似乎需要不同的冲突消解策略。**开放的问题是：在特定情境下，最佳策略之间是否存在某种相关性？**我们预期，用户模型维度的等价类本身就与最佳策略相关。如果确实如此，这个信息可以被加进通用用户模型本体 GUMO，于是这个问题可以通过引入一层新的元规则来解决。

6.3.4 检索结果的冲突排序

If several statements are returned, the `query.ranking` attribute controls the order of the statements. They show that the same terms are used in the `query.ranking` and in the `query.strategy` attribute, however with a different impact.

如果返回了多条陈述，`query.ranking` 属性控制这些陈述的顺序。**要注意的是，`ranking` 属性和 `strategy` 属性用的是同一批词，但作用不同。**

（原文图 6.17 给出三个排序示例：一、按最近排。二、先按最近排、再按最可信排。三、先按说得最多排、再按最具体排、再按最贴本人排。）

The first ranking orders the resulting statements according to a temporal axis, the second ranking adds to the temporal axis the second level ordering on a confidence axis, and the third

example ranking orders the statements under completely different settings.

第一个排序按时间轴给结果陈述排序；第二个排序在时间轴之外，又加了一层按置信度轴的次级排序；第三个示例排序则在完全不同的设定下给陈述排序。

6.3.5 语义冲突集的检测与分类

In the prefacing chess example the conflict was easy to detect since all three statements used the same subject, the same auxiliary, the same predicate and even the same range. If one statement used instead the range poor-average-good-excellent a so-called SEMANTIC RANGE MAPPING had to be executed, where for example poor is mapped to low, average and good are mapped to medium, and excellent is mapped to high.

在开篇那个下棋的例子中，冲突很容易被检测出来，因为三条陈述用的是同一个主语、同一个辅助词、同一个谓词，甚至同一个取值域。如果其中一条改用「差-中等-好-优秀」这个取值域，就必须执行所谓的**语义刻度映射**：比如把「差」映射到「低」，把「中等」和「好」映射到「中」，把「优秀」映射到「高」。

A more tricky problem is how can we detect conflicts if even the auxiliary and predicate in the conflicting statements differ even though the same ontology is used? For example, system B could use auxiliary=hasKnowledge and predicate=chess while system C uses auxiliary=hasAbility and predicate=boardGames. In such a case, we already need SEMANTIC PROPERTY MAPPING. A closely related problem arises if statements refer to different ontologies. In such a case we additionally need ONTOLOGY MAPPING.

更麻烦的问题是：即使用的是同一个本体，如果互相矛盾的陈述连辅助词和谓词都不一样，我们该怎么检测出冲突？举例来说，系统 B 可能用「有知识」这个辅助词加「国际象棋」这个谓词，而系统 C 用「有能力」这个辅助词加「棋类游戏」这个谓词。这种情况下我们就需要**语义属性映射**了（这里「属性」指辅助词与谓词这一对）。还有一个紧密相关的问题：如果这些陈述引用的是不同的本体，那我们还额外需要**本体映射**。

For the challenge of detecting possible conflicts we use a technique which is also used in production systems: we classify statements into so-called conflict sets. Mathematically speaking these statements are classified into equivalence classes. The following listing analyzes several types of such equivalence classes and arranges them into six levels with increasing computational load.

为了应对「检测可能的冲突」这个挑战，我们采用一种在产生式系统中也在用的技术：把陈述分类进所谓的**冲突集**。用数学的说法，就是把这些陈述分类进等价类。下面的清单分析了几种这样的等价类，并把它们按计算负载递增排成六个级别。

Class 0: Statements with the same subject, auxiliary, predicate and range are called to be in a potential conflict of level 0. These conflicts are easy to detect since only a syntactical comparison is necessary. However, every fast changing user model dimension like heart beat for

example would constantly produce potential conflicts and thus the produced conflict sets are not very useful.

第 0 级：主语、辅助词、谓词、取值域四项都相同的陈述，被称为处于第 0 级潜在冲突。这类冲突容易检测，因为只需要做语法比较。**但是像心跳这种快速变化的维度会不断产生潜在冲突，因此产出的冲突集用处不大。**

Class 1: The problem of class 0 is solved by removing all statements that are already expired or replaced. The following simple conflicts that only differ in the object value can be detected: Tim is happy = yes / Tim is happy = no.

第 1 级：把所有已过期或已被替换的陈述去掉，就解决了第 0 级的问题。像下面这种只在宾语取值上不同的简单冲突就能被检测出来：Tim 快乐 = 是；Tim 快乐 = 否。

Class 2: The conflict sets only match against subject, auxiliary and predicate while the ranges are transformed into each other with the SEMANTIC RANGE MAPPING. The mappings between different ranges are defined in the GUMO ontology. Example: Tim happy range=yesNo object=yes / Tim happy range=lowHigh object=low.

第 2 级：冲突集只对主语、辅助词、谓词做匹配，而取值域之间用语义刻度映射互相转换。**不同刻度之间的映射定义在 GUMO 本体里。**例子：Tim 快乐，刻度是「是/否」，取值「是」；Tim 快乐，刻度是「低/高」，取值「低」。

Class 3: The conflict sets can differ in the predicate, range and object values. Related predicates are classified together. This relation could for example be defined as synonyms (other predicates with the same meaning) or hypernyms (concepts with a broader meaning) or holonyms (the whole concept of which the predicate is part of) or opponyms (concepts with opposite meanings like happy versus sad or extraverted versus introverted). Example: Tim happy=yes / Tim sad=high.

第 3 级：冲突集在谓词、取值域和宾语取值上都可以不同，相关的谓词被归到一起。这种相关关系可以定义为同义词（含义相同的其他谓词）、上位词（含义更宽的概念）、整体词（谓词所属的那个整体概念）、或者反义词（含义相反的概念，比如快乐对悲伤、外向对内向）。例子：Tim 快乐 = 是；Tim 悲伤 = 高。

Class 4: A slight extension to level 3, where additionally the auxiliary attributes are semantically related. For example hasLearned and hasKnowledge are closely related. Also, if someone hasInterest in something, one can assume that at least a rudimentary hasKnowledge statement also holds. Furthermore if a hasPreference relation holds, a hasInterest relation should also hold. However, this level 4 has not yet been used for concrete conflict resolution.

第 4 级：只是对第 3 级的小幅扩展，额外允许辅助词之间在语义上相关。举例来说，「已学过」和「有知识」是紧密相关的。另外，如果某人对某事「有兴趣」，可以假定至少一条初步的「有知识」陈述也成立。再者，如果「有偏好」这个关系成立，那么「有兴趣」这个关系也应当成立。**不过第 4 级尚未被用于具体的冲突消解。**

Class 5: Interesting about level 5 is that the statements can completely differ syntactically but still be recognized as being semantically conflicting. The newly extended subject integrates the modeling of user groups. If the subject defines a group or is member of a group, the corresponding GROUP MEMBER MAPPING has to be applied in order to handle conflicts between "member-statements" and "group-statements". Furthermore it allows to handle the problem that one user is often identified in different systems with different logins or nicknames.

第 5 级：第 5 级有意思的地方在于，这些陈述在语法上可以完全不同，却仍然能被识别为语义上互相矛盾。被扩展的主语这一项整合了用户群组的建模：如果主语定义了一个群组、或者是某个群组的成员，就必须应用相应的**组成员映射**，以处理「成员陈述」与「群组陈述」之间的冲突。此外它还能处理这样一个问题：同一个用户在不同系统里常常用不同的登录名或昵称来标识。

（原文脚注举了一个真实的多身份例子：在一个已实现的普适用户建模场景里，定位系统用设备的 IP 做标识，UbisWorld 用自己内部的编号，地图工具用 URI 建模，而购物助手又用另一套本体表示。）

To summarize, the complex task of detecting conflicting statements in a repository can be analyzed with a fine-grained definition of conflict sets, that represent syntactical as well as semantical conflicts. Ontological reasoning is mostly integrated by using semantic mapping functionality. However, further research has to be done at this point.

总结起来，「在一个库里检测互相矛盾的陈述」这项复杂任务，可以通过对冲突集做细粒度的定义来分析，这些冲突集既表示语法冲突也表示语义冲突。本体推理主要是通过语义映射功能被整合进来的。**不过这一处还需要进一步的研究。**

6.3.6 智能情境检索的架构

（原文图 6.18 给出智能情境检索的架构图，重点在语义冲突消解部分，它代表查询过程里的控制这个宏步骤。图上用带圈的数字标出阅读顺序：）

一、用 UserQL 写的请求进来，先被解析。二、分布式地检索情境陈述。三、汇总选择、匹配、过滤三个宏步骤，把过滤结果作为冲突消解过程的输入。四、三个语法层面的处理：变体映射、移除已过期、移除已被替换。五、三个语义层面的处理：组成员映射、语义属性映射、语义刻度映射。**这三者建立在 GUMO 本体、UbisWorld 本体、SUMO/MILO 本体，以及 WordNet 这个知识库之上。**六、检测冲突，并构造出「主语、辅助词、谓词、取值域四项都做语义扩展，且排除已过期与已被替换」的冲突集。七、后处理：排序、格式、命名、函数，这四项控制输出形态。八、形成最终的 UserML 报告，通过 HTTP 发给请求方。

6.4 讨论与设计决定

One design idea behind the user model query language UserQL was that it should be syntactically simple (realized as a flat structure) and close to the user model markup language UserML. The knowledge about the functionality should analogously be defined in an ontology such that the semantics of the queries' attributes and the statements' attributes are uniform.

However, the most important idea was to integrate syntactic and semantic conflict resolution into the query language.

用户模型查询语言 UserQL 背后的一个设计想法是：它在语法上应当简单（实现成一个扁平结构），并且贴近用户模型标记语言 UserML。关于功能的知识同样应当定义在本体里，使得查询属性的语义与陈述属性的语义保持统一。**不过最重要的想法是把语法层面和语义层面的冲突消解整合进查询语言本身。**

Even though the so-called SMART SITUATION RETRIEVAL integrates various techniques of information retrieval and ontological reasoning, the dominating point of view should still be information retrieval and not the ontology. That is to say the query process which is mostly applied to databases with SITUATIONAL STATEMENTS dominates in our approach the ontological inference.

尽管所谓的「智能情境检索」整合了信息检索与本体推理的多种技术，**占主导地位的视角仍然应当是信息检索，而不是本体。**也就是说，在我们这套方法里，主要作用在装着情境陈述的数据库上的查询过程，压倒了对本体推断。

Our situation models, that are part of the A-boxes, may be full of inconsistencies that need not to be resolved in general since we do not intend to change the statement repositories into one consistent world model. The statements that have been collected and sent around won't be changed or deleted. The reason is that in the highly distributed architecture information is passed on and in most cases, we don't have access rights to edit the various instances of the statements.

我们的情境模型属于 A-box（实例层）的一部分，**它们可能充满不一致，而这些不一致一般并不需要被解决——因为我们并不打算把陈述库变成一个前后一致的世界模型。**已经被收集并传播出去的陈述不会被修改或删除。理由是在这个高度分布的架构里，信息是被层层传递的，而多数情况下我们没有编辑这些陈述各个副本的访问权限。

However, we allow to comment on them by reifying a statement into a new one and we allow a statement to be marked as being replaced. Furthermore we allow hiding them with the integrated privacy mechanism. The first advantage is that each user and each requesting system can individually choose the preferred conflict strategies and receive its individual model. The second advantage is that the integration of partial, decentralized user models can directly be realized using the SMART SITUATION RETRIEVAL. The third advantage is that privacy handling is not the only mechanism that leads to individual models which results in better privacy.

不过我们允许对这些陈述做评论——把一条陈述具体化成一条新的陈述；也允许把一条陈述标记为「已被替换」；此外还允许用内建的隐私机制把它们隐藏起来。**第一个好处是每个用户、每个请求方都能各自选择偏好的冲突策略，从而得到属于自己的那份模型。**第二个好处是，局部的、分布式的用户模型的集成，可以直接用智能情境检索来实现。第三个好处是，导致「个体化模型」的机制不只有隐私处理这一个，而这带来了更好的隐私性。

The challenge that different users and different systems may want to use individualized ontologies that control the inferences could be solved in the following way: a new attribute with the name ontology is added to the Control Box that carries one or several URIs pointing to the semantic web representations of these ontologies. This means that not only the set of statements differ individually and the conflict resolving strategies differ individually but also the inference ontologies can be chosen individually.

「不同用户和不同系统可能想使用各自定制的、控制推断的本体」这个挑战，可以这样解决：往控制盒里新加一个名叫 `ontology` 的属性，它携带一个或多个指向这些本体的语义网表示的地址。这意味着不仅陈述集合可以因人而异、冲突消解策略可以因人而异，**连推断所用的本体也可以因人而异地选择。**

Another issue: how should we apply for example a confidence filter, if several statements in a conflict set don't carry confidence values? Or how should we apply the recency filter if the temporal constraints are not given? The answer is that the conflict resolution can only be as good as the available set of statements.

还有一个问题：如果冲突集里有几条陈述根本没带置信度值，我们该怎么应用置信度过滤器？或者如果时间约束没有给出，我们该怎么应用时效过滤器？**答案是：冲突消解的好坏，只能与可用的那批陈述一样好。**

参考链接

- [Ubiquitous User Modeling（本书条目）](#) — 本章出自这本 2006 年出版的博士论文，DFKI 的出版物页面
- [WordNet](#) — 第 3 级及以上语义冲突检测所依赖的词汇关系资源（同义、上位、整体、反义）
- [SUMO 上层本体](#) — 架构图第五步里语义映射所依赖的上层本体之一